
PDAGENT-BENCH: Characterizing, Grounding, and Architecting LLM/VLM Agents for VLSI Physical Design

Qiufeng Li^{1,*}, Rongqian Chen¹, Quan Cheng², Chengxuan Wang³, Sizhe Tang¹

Chia-Tung Ho⁴, David Z. Pan⁵, Tian Lan¹, Weidong Cao^{1,*}

¹ George Washington University ² Brown University ³ University of California, Los Angeles

⁴ NVIDIA ⁵ The University of Texas at Austin

{qiufengli, weidong.cao}@gwu.edu

Abstract

Large Language Models (LLMs) and vision-language models (VLMs) have shown remarkable success in the front-end design of Very Large-Scale Integrated Circuits (VLSI), yet their capabilities for VLSI physical design remain significantly underexplored. One primary cause is the lack of standardized benchmarks for evaluating agentic physical design workflows that require high-dimensional, multi-stage optimization under strict design constraints, coordinated interaction with diverse Electronic Design Automation (EDA) tools, and iterative refinement. This work introduces **PDAGENT-BENCH**, a comprehensive and multi-dimensional **benchmark** for evaluating LLM/VLM-based **agents** across the **physical design stack**. PDAGENT-BENCH integrates both task-level assessment and workflow-level execution. The benchmark suite contains 353 curated problems that combine conceptual questions with real-world industrial artifacts, with expert-validated references and executable solutions. **These tasks cover five key capability dimensions:** foundational knowledge, report comprehension, root-cause analysis, script generation, and full-flow implementation (including **macro placement**). In addition, the benchmark provides **a unified, human-aligned agentic physical design workflow framework** that enables closed-loop evaluation of holistic physical design in realistic EDA environments (**by preserving privacy when integrating commercial EDA tools, PDKs, and frontier LLMs/VLMs**). Experiments on 11 state-of-the-art models reveal that while modern LLMs/VLMs perform competitively on conceptual tasks (e.g., GPT-5.5 achieves 73.3% on root-cause analysis), they remain substantially limited in tool-centric execution and long-horizon, multi-stage reasoning. Our studies further show that human-skill-enhanced agentic workflows significantly improve end-to-end physical design performance. Specifically, **with human instructions, our experiments demonstrate the superior capabilities of VLM in macro placement beyond state-of-the-art analytical and reinforcement learning-based methods**. PDAGENT-BENCH establishes a standardized, reproducible, and realistic evaluation framework for advancing LLM/VLM-driven holistic physical design automation¹. To ensure full reproducibility and broad accessibility, we will release PDAGENT-Bench together with its agentic workflow framework, instantiated on open-source PDKs (e.g., Nangate45, ASAP7) and open EDA tools (e.g., OpenROAD).

¹All findings and conclusions presented in this work are based solely on the authors' independent experiments and analyses, and do not reflect or imply any views, opinions, or endorsements from industry.

1 Introduction

Very Large-Scale Integrated Circuits (VLSI) are foundational hardware to power various modern technological advances, such as AI, scientific computing, data centers, and more [35, 14]. The explosive growth of applications in these critical sectors is driving an unprecedented demand for VLSI design with higher productivity, better performance, and greater scale. Yet, conventional VLSI design workflows are increasingly unable to keep pace (Figure 1). A fundamental limitation lies in the **inherently iterative and human-in-the-loop design paradigm**, where engineers must repeatedly orchestrate, analyze, and refine design decisions across the entire stack, from front-end synthesis to back-end physical design [41]. This tightly coupled, multi-stage workflow results in substantial engineering overhead, long feedback cycles, and limited scalability, particularly as design complexity continues to grow [16]. Recent advances in large language models (LLMs) and vision-language models (VLMs) have opened a promising direction toward *agentic design automation*, in which LLM/VLM-based agents can iteratively reason, act, and optimize solutions in closed-loop settings. Specifically, in front-end design, LLMs have demonstrated strong capabilities in generating and debugging hardware code, significantly reducing manual effort [13, 44, 19]. For example, NVIDIA’s ChipNeMo [19] shows that high-quality RTL (Register-Transfer Level) code can be generated with just a few prompts. Importantly, these advances have not emerged in isolation; rather, they have been enabled and sustained by **well-defined benchmarks and evaluation frameworks** [25, 22, 20], which provide standardized tasks, reproducible protocols, and quantitative metrics for assessing progress.

Yet, **physical design remains largely underexplored with LLMs/VLMs**, despite its decisive role in determining a chip’s final power, performance, and area (PPA). The primary barrier to progress is the **lack of standardized benchmarks** that can capture the full complexity of physical design and enable systematic evaluation of LLM/VLM-based agentic physical design workflows. It is important to note that physical design presents fundamentally different challenges from front-end tasks: **physical design involves complex geometric constraints, strict technology design rules, multi-tool orchestration, and long-latency feedback loops**. Thus, it requires integrating diverse cognitive capabilities: understanding design rules, interpreting tool-generated reports, diagnosing violations, generating tool scripts, and coordinating iterative optimization across stages. Existing benchmarks fail to capture this complexity. Prior works such as ChiPBench [34] focus on isolated subproblems (e.g., macro placement) with conventional algorithms, while ChatEDA-Bench [38] and iScript-Bench [39] evaluate narrow capabilities such as script generation. As a result, **current benchmarks are fragmented and disconnected from end-to-end workflows**, leaving them insufficient to assess whether LLM/VLM agents can effectively support realistic physical design.

In this work, we seek to evaluate the capabilities of LLM/VLM-based agents that are engineered to enable *holistic physical design* in realistic EDA environments. Formally, this pursuit requires assessing an agent’s ability to (i) perform various types of tasks throughout the design stack, (ii) operate within multi-stage tool-driven workflows, and (iii) optimize design objectives (e.g., PPA) through iterative interaction with EDA tools. Achieving the goal requires a benchmark that satisfies several key desiderata. **(D1) Coverage.** It must span the full spectrum of physical design capabilities, beyond isolated tasks. **(D2) Workflow Fidelity.** Evaluation must reflect realistic multi-stage workflows with tool-in-the-loop feedback. **(D3) Standardization.** Tasks, inputs, and metrics must be reproducible and comparable across models. **(D4) Compositionality.** The benchmark should capture interactions between capabilities (e.g., report understanding → debugging → script generation → tool use). **(D5) Diagnostic Ability.** It should enable fine-grained analysis of failure modes and capability gaps.

To this end, we introduce **PDAGENT-BENCH**, a first-of-its-kind **benchmark for holistic and reproducible evaluation of LLM-based agentic physical design**. Our benchmark integrates both task-level evaluation and workflow-level execution. **Benchmark Design.** PDAGENT-BENCH consists of a **multi-dimensional benchmark suite** with 353 curated tasks spanning five capability dimensions: (i) foundational knowledge understanding, (ii) report comprehension, (iii) root-cause analysis, (iv) script generation, and (v) full-flow orchestration. Each task is designed to reflect realistic scenarios in physical design and is paired with standardized evaluation criteria. **Evaluation Framework.** We further develop a **unified human-aligned agentic design workflow framework** that enables closed-loop interaction between LLM/VLM agents and various EDA tools. This framework allows agents to iteratively **place macros**, generate actions (e.g., scripts), receive tool feedback (e.g., timing or congestion reports), and refine decisions, thereby enabling evaluation under realistic design loops rather than static prompts. Using this framework, we conduct a comprehensive evaluation on 11

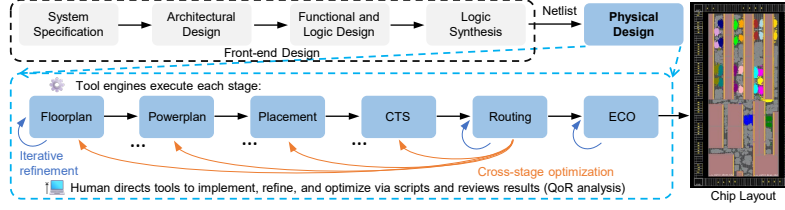


Figure 1: Modern VLSI design workflow. It spans from system specification through physical design, with physical design encompassing floorplan through engineering change order (ECO). Each stage involves iterative refinement within the stage (blue) and may trigger cross-stage optimization that loops back to earlier stages (orange). EDA tool engines execute each stage, while human engineers direct the tools via scripts and constraints, and review results with quality-of-results (QoR) analysis.

state-of-the-art LLMs/VLMs. Our results provide the first systematic characterization of LLM/VLM capabilities in physical design, revealing critical limitations in long-horizon reasoning, tool interaction, and cross-stage coordination.

Our key contributions are summarized as follows: (i) We propose **PDAGENT-BENCH**, the first benchmark for **end-to-end agentic VLSI physical design workflows**, enabling standardized and reproducible evaluation. (ii) We construct a **353-task benchmark suite** covering five key capability dimensions, bridging isolated tasks and holistic workflows. (iii) We develop a **unified agent-in-the-loop framework** for closed-loop evaluation with EDA tools by plugging in existing LLMs/VLMs. (iv) We present a **systematic empirical study** on 11 LLMs/VLMs, **showing great potential of VLMs in macro placement and identifying key capability gaps, failure modes, and critical insights for domain-specific model development**.

2 Background and Related Work

VLSI Design Workflow. A typical VLSI design workflow used by human designers is shown in Figure 1, which has two phases: *front-end design* and *physical design*. The front-end phase includes architectural design, functional and logic design (hardware coding), and logic synthesis, where high-level functional specifications are translated into a gate-level netlist. This netlist serves as an input to the physical design phase, which generates a manufacturable layout. Physical design is a **multi-stage, tightly coupled optimization process** comprising several key steps [16]. **Floorplanning** defines the spatial organization of macros and functional blocks, including core area, I/O pin locations, and macro placement, often requiring substantial manual guidance due to limited automation in current tools. **Power planning** constructs a hierarchical power delivery network (PDN) (e.g., rings, stripes, meshes, vias) to satisfy IR-drop and electromigration constraints while minimizing routing overhead. **Placement** assigns legal locations to standard cells, optimizing objectives such as wirelength, congestion, timing, and density. **Clock Tree Synthesis (CTS)** builds a buffered clock network to distribute the clock signal with controlled skew, latency, and transition characteristics. **Routing** establishes signal connectivity across metal layers under strict design-rule constraints, balancing wirelength, congestion, and manufacturability. **Engineering Change Order (ECO)** performs localized post-route modifications, including cell resizing, buffer insertion, and metal-only fixes, to resolve residual timing violations and design-rule errors without disrupting the global layout. Although each stage is supported by specialized EDA tools, achieving high-quality designs still relies heavily on **human experts to orchestrate the tools in the design workflow**. Importantly, these stages are **strongly interdependent**: early decisions (e.g., floorplanning) constrain downstream feasibility; placement impacts congestion and timing closure; and both influence the effectiveness of CTS, routing, and ECO. As a result, physical design requires **iterative, cross-stage refinement under long feedback loops**, making it a complex, feedback-driven process that is difficult to automate and scale.

Agentic Modeling and Benchmarking. LLM/VLM-based agents are well-suited for VLSI physical design due to two key properties in the physical design workflow. (i) **Script-driven, tool-mediated interaction.** Physical design stages are executed via parameterized tool commands (e.g., Tcl scripts), which encode key decisions such as PDN configuration, placement constraints, and CTS strategies. Consequently, the workflow can be abstracted as a sequence of *state-action-feedback* interactions, where an agent observes design states (e.g., reports, constraints), generates executable actions

Table 1: Comparison of PDAGENT-BENCH with other circuit benchmarks.

Benchmark Features	VerilogEval V2 [25]	CircuitNet 3.0 [33]	ChipBench [34]	ChatEDA-Bench [38]	PDAGENT-BENCH (Ours)
Open Source	✓	✓	✓	✓	✓
RTL-to-GDS Flow	×	×	✓	✓	✓
Industrial Setting [†]	×	✓	×	×	✓
Domain Knowledge	×	×	×	×	✓
Report Analysis	×	×	×	×	✓
Root Cause Analysis	×	×	×	×	✓
Static Timing Analysis	×	×	×	×	✓
# of Data	153	8,659	20	50	353
Target Tasks	RTL Generation	Early-Stage Timing/Power Prediction	Macro Placement Optimization	Script Generation	Comprehensive Physical Design (including macro placement)

[†] **Industrial Setting:** uses commercial EDA tools on advanced technology nodes (≤ 28 nm).

(scripts), and receives feedback from tools. This aligns closely with the LLM/VLM agent paradigm of reasoning, tool invocation, and iterative refinement. **(ii) Structural regularity and compositional skill reuse.** Physical design exhibits strong regularity across designs within the same circuit class (e.g., CPUs, accelerators). Common patterns arise in floorplanning templates, PDN topologies, placement strategies, and timing closure heuristics. In practice, expert designers leverage this regularity by reusing and adapting prior solutions rather than solving each instance from scratch. This suggests that effective automation requires not only reasoning, but also **retrieval and composition of reusable skills** (e.g., script templates, optimization procedures). Such properties make physical design a natural setting for evaluating **generalization, compositionality, and skill reuse** in LLM agents. Together, these properties make physical design a natural setting for **closed-loop, tool-integrated evaluation** of LLM agents, motivating the need for a holistic benchmark for agentic physical design.

Related Work. Recently, several benchmarks [22, 20, 25, 33, 34, 38, 39, 21, 11] have been proposed to evaluate LLM/VLM-based agents and conventional optimization algorithms for VLSI design, as summarized in Table 1. VerilogEval v2 [25] is restricted to RTL code generation, while CircuitNet 3.0 [33] is confined to early-stage timing and power prediction. ChipBench [34] focuses on macro-placement algorithms (analytical methods + reinforcement learning), and ChatEDA-Bench [38] and iScript-Bench [39] primarily evaluate EDA script generation. Despite these contributions, existing benchmarks share a key limitation: they evaluate **isolated capabilities** rather than **end-to-end workflows**. Beyond benchmarks, recent agent methods [12, 37, 27, 26] such as ORFS-agent [12] and OpenROAD Agent [37] demonstrate closed-loop interaction with EDA tools, iteratively tuning flow parameters and generating self-correcting scripts from tool feedback, precisely the capabilities that existing benchmarks fail to capture. However, none of the current physical design benchmarks capture the closed-loop interaction between agents and EDA tools, nor do they assess critical capabilities such as report comprehension, diagnostic reasoning, and iterative refinement across stages. As a result, they fail to reflect the **feedback-driven, multi-stage nature** of real-world physical design, leaving a significant gap in the evaluation of whether LLM/VLM agents can operate effectively in practical settings. PDAGENT-BENCH addresses this critical gap by evaluating all these dimensions jointly (Table 1).

3 PDAGENT-BENCH

PDAGENT-BENCH consists of two complementary components: (i) a *multi-dimensional benchmark dataset* that captures the breadth and complexity of physical design tasks, and (ii) a *tool-integrated, multi-agent design workflow framework* that enables evaluation under realistic EDA interactions. Together, these components support **holistic, standardized, and reproducible evaluation** of LLM/VLM agents in holistic physical design.

3.1 Benchmark Development

Overview. To systematically evaluate LLM/VLM capabilities in physical design, we construct a benchmark of 353 curated problems (Table 2). The dataset spans a spectrum from foundational knowledge questions to production-oriented tasks, including report comprehension, root-cause analy-

Table 2: Overview of benchmark categories used to evaluate agents on physical design tasks. Report Comprehension, Root Cause Analysis, and Static Timing Analysis are multi-modal tasks that pair text with EDA reports, layouts, or timing diagrams. The targeted agent capabilities are denoted by: ① Reasoning, ② Planning, ③ Perception, ④ Tool Use.

Task	Category	#	Description	Agent Capability
Physical Design Understanding	Foundational Knowledge	90	Conceptual questions on physical-design and EDA fundamentals (timing, power, placement, routing, DRC/LVS).	①, ②
	Report Comprehension	11	Interpreting EDA tool reports (timing, congestion, power, utilization) to extract metrics and identify issues.	①, ③
	Root Cause Analysis	21	Diagnosing the underlying cause of PPA or convergence problems and proposing optimization directions.	①, ③
	Static Timing Analysis	11	Path-level reasoning over setup/hold violations, slack, clock skew, and timing exceptions.	①, ③
Script Generation	P&R Vendor 2	90	Generating Cadence Innovus TCL scripts for place-and-route flow steps.	④
	P&R Vendor 1	88	Generating Synopsys IC Compiler II TCL scripts for backend implementation.	④
	ECO	10	Generating engineering-change-order scripts for post-route fixes (timing, DRC, functional).	④
	FM (Formal Verification)	22	Generating Synopsys Formality scripts for RTL-vs-netlist equivalence checking.	④
Full Flow Implementation	End-to-End placement and route (PnR)	10	Driving a complete RTL-to-GDSII flow on the design suite below, evaluated on PPA, timing closure, and DRC/LVS cleanliness.	①, ②, ③, ④

Table 3: Benchmark designs used in the full-flow implementation. We report the cell count after synthesis, the target clock period, and the utilization target.

Design	Description	#Cells	Target Clk (ns)	Util.
TinyRISC-V [17]	Compact RISC-V core (small control-logic design).	15,012	5.0	0.65
Open910 [6]	Open-source application-class processor.	754,981	1.5	0.60
Systolic array [1]	Matrix-multiply accelerator (datapath-heavy).	41,198	3.4	0.50
16b pipelined mult. [24]	16-bit pipelined multiplier (arithmetic datapath).	342	2.6	0.55
AES-256 [15]	AES-256 cryptographic core.	15,516	3.5	0.60
NVDLA-small [23]	NVIDIA Deep Learning Accelerator, small config.	270,072	2.0	0.60
NVDLA-large [23]	NVIDIA Deep Learning Accelerator, large config.	1,478,865	2.5	0.60
UART controller [32]	Serial communication peripheral.	368	3.4	0.65
Elevator FSM [24]	Finite-state-machine control design.	176	2.5	0.70
Ethernet MAC [24]	Ethernet media access controller.	35,172	10.0	0.60

sis, optimization reasoning, and script generation. This design aims to capture the core competencies required of a physical-design engineer while enabling fine-grained analysis of model performance.

Data Selection Criteria. Each problem in our benchmark dataset is selected to satisfy the following principles. **(C1) Coverage and diversity.** Tasks span multiple stages of the physical design flow and require both domain knowledge and structured reasoning. **(C2) Ground-truth fidelity.** Each instance is paired with a detailed reference solution to enable precise evaluation and error analysis. **(C3) Multi-modality.** Problems incorporate text, scripts, and visual elements (e.g., floorplans, design rule check (DRC) maps), reflecting real-world inputs. **(C4) Leakage resistance.** Instances are curated to minimize overlap with publicly available data, reducing the risk of memorization-based performance. **(C5) Realistic difficulty.** Tasks extend beyond simple question-answering (QA) to include complex operations such as timing report interpretation and full-flow reasoning.

Data Representation and Preprocessing. The raw dataset comprises 3 modalities, including scripts, free-form text, and images. To enable consistent processing and evaluation, we normalize all entries into a **unified JSON schema**, ensuring structured representation and efficient parsing. For **script generation tasks**, we provide a comprehensive set of acceptable solutions, accounting for multiple valid command variants due to tool-version differences, alternative syntactic forms, and synonymous expressions. For **question-answering tasks**, we design detailed evaluation rubrics with clearly defined scoring criteria and key points, enabling reliable and consistent **LLM-as-judge** assessment.

Benchmark. Following the criteria and preprocessing strategies, we construct PDAGENT-BENCH benchmark dataset, summarized in Table 2. The benchmark comprises two complementary compo-

nents. **First, foundational suite (90 instances).** We curate 90 foundational questions that cover the full physical design flow, from initialization to tape-out, ensuring broad coverage of core concepts and design stages. **Second, production-oriented suite (263 instances).** To evaluate performance on real-world, production-oriented tasks, we include 263 additional instances derived from commonly used implementation scripts, timing reports, and error/violation reports collected from our projects, whose results have been published at top-tier solid-state circuit design conferences including ISSCC [10], VLSI Symposium [9], A-SSCC [7], and CICC [8]. The script generation spans the major commercial EDA tools used across the modern physical-design flow. To ensure quality and reliability, all questions and reference answers are independently reviewed by three domain experts. In addition, all script-based solutions are validated through execution, confirming functional correctness and practical usability within a simulation environment. We organize the benchmark into three evaluation suites. The **Physical Design Understanding suite (133 instances)** evaluates foundational concepts, EDA report comprehension, root-cause analysis, and static timing analysis. The **Script Generation suite (210 instances)** tests executable Tcl generation across the major commercial EDA tools. The **Full-Flow Implementation suite** (Table 3) drives a complete RTL-to-GDSII (physical layout) flow on 10 designs, ranging from a 176-cell FSM (finite state machine) to a 1.5M-cell NVDLA accelerator, graded on placement quality, timing closure, DRC (design rule check)/LVS (layout vs. schematic) cleanliness, and PPA. As summarized in Table 2, **our benchmark is designed to probe the key capabilities required of agents, spanning four core aspects: planning, reasoning, tool use, and perception** [42].

3.2 Agentic Design Workflow Framework

Overview. To enable systematic benchmarking of human-aligned agentic workflows for VLSI physical design, we develop **PDAGENT**, a unified framework for constructing and evaluating LLM/VLM-based agents in closed-loop, tool-integrated environments (Figure 2(a)). The framework defines standardized **agent roles, interfaces, and coordination mechanisms**, allowing researchers to instantiate and compare diverse agentic workflows under consistent conditions. It is designed to be **modular and extensible**, supporting plug-and-play integration of different models (e.g., open-source or proprietary LLMs/VLMs) and customization for varying design objectives. In particular, our framework adopts a privacy-preserving approach to integrate commercial EDA tools, PDKs, and LLMs/VLMs through a local environment (Section A.9). This corresponds to partially addressing interoperability and deployment issues due to the multi-party involvement, an important domain-specific gap raised in the recent Cognitive Silicon Design Workshop (CSDW) [36].

Given a gate-level netlist, technology constraints, and design targets, PDAGENT orchestrates agents to iteratively produce a DRC-clean, timing-closed layout. The workflow follows the natural structure of physical design, comprising four phases, *planning, implementation, debugging, and optimization*, which form a closed loop for progressive refinement. Specifically, PDAGENT comprises a Planner Agent, Worker Agent, Debugger Agent, and Optimizer Agent, augmented by an Analyzer Agent that extracts key metrics from verbose EDA tool logs. We briefly introduce the key design knobs of PDAGENT below. Detailed descriptions of each agent’s role are provided in Appendix A.6.

Agent Roles. The PDAGENT comprises five agents, each encapsulating a distinct concern: (i) *Planner* serves as the central hub, decomposing user intent into a directed acyclic graph (DAG) of subtasks, routing all inter-agent messages, and maintaining global execution; (ii) *Worker* interfaces with the EDA tool (e.g., Innovus) via a persistent interactive shell, executing TCL commands and returning structured results, illustrated in Figure 2(b). Notably, **macro placement during the floorplan stage is performed directly by the NLMs agents**, which takes the layout image as visual input to guide placement decisions (see Appendix A.6.1 for details); (iii) *Analyzer* extracts quantitative metrics, timing, for example, worst negative slack (WNS), physical (DRC, utilization), and power, from EDA logs; (iv) *Debugger* diagnoses failures through LLM/VLM-powered root-cause analysis; (v) *Optimizer* performs parameter tuning guided by metric trajectories and domain knowledge, as shown in Figure 2(c).

Hub-and-Spoke Communication. All inter-agent messages are routed through the Planning Agent, which maintains full observability of the system state and enforces execution ordering. The sole exception is *tool interaction*: any agent may directly invoke EDA tool commands via a synchronous `execute_tool` interface, bypassing hub routing to avoid latency overhead for frequent tool queries. This hybrid topology balances centralized coordination with efficient tool interaction.

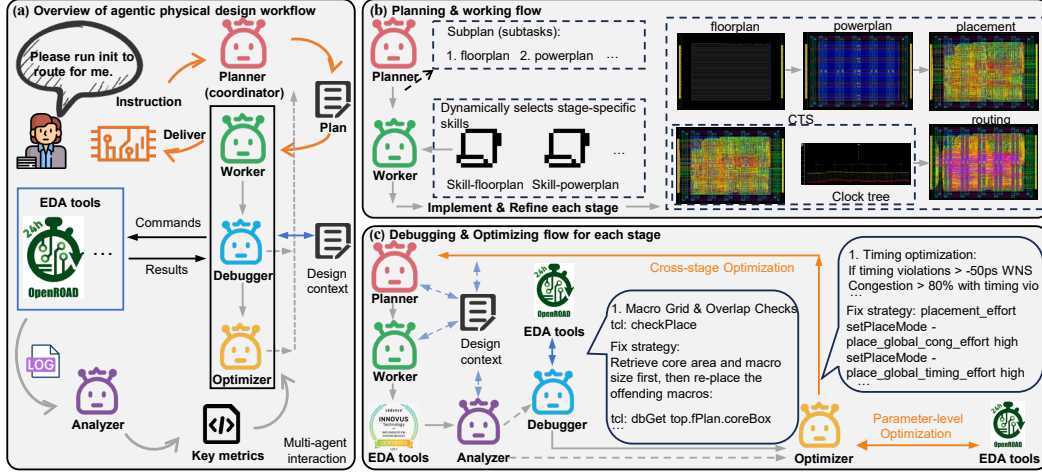


Figure 2: (a) Overview of the PDAgent framework. Five specialized agents (Planner, Worker, Debugger, Optimizer, and Analyzer) coordinate to translate user instructions into a complete physical-design flow. (b) The Planner decomposes the task into stage-wise subplans, and the Worker invokes stage-specific skills for each stage, advancing only after debugging and optimization are complete. (c) The Optimizer and Debugger refine the design through targeted fix strategies until convergence.

Skill Packs. Building on the inherent skill-driven physical design workflow introduced in Section 2, the agentic framework is *grounded* in a curated skill library that encodes the procedural knowledge of senior engineers, yielding a skill-enhanced LLM/VLM-based agentic framework. Skills encode procedural knowledge, executable scripts together with the rationale behind parameter choices and PPA optimization strategies, distilled from senior engineers. This kind of tacit, experience-driven expertise is what traditional algorithmic approaches cannot capture, and what motivates an LLM/VLM-based agentic framework. Domain knowledge is organized into *skill packs*, self-contained directories structured as `skills_{tech}_{tool}/` (e.g., `skills_tsmc28nm_innovus/`). Each skill pack contains per-stage skill files that encode expert knowledge: reference TCL scripts, parameter-tuning strategies, debug decision matrices, and cross-stage impact analyses. Skills are auto-discovered at initialization and resolved dynamically via fuzzy matching, enabling seamless extension to new technology nodes and EDA tools without modifying agent code.

Closed-Loop PPA Optimization. Aligning with the practical VLSI design, the optimization mechanism operates at two levels: a **stage-level** loop that progresses through the physical design flow step by step, and a **parameter-level** loop that explores the design space within each stage. When design targets are unmet, the agent employs a **tree-of-thought exploration strategy**, generating and evaluating multiple candidate actions before selecting the most promising trajectory. Iteration continues until convergence or a predefined budget is reached.

4 Experiments and Results

This section presents comprehensive evaluations of various LLM/VLM-based agents on VLSI physical design tasks by using PDAGENT-BENCH. We first describe the experimental setup, then report the main results, and finally summarize key findings and insights.

4.1 Experimental Setup

Models and Computing Platforms. We evaluate 11 LLMs/VLMs with PDAGENT-BENCH, including 6 proprietary and 5 open-source models. The proprietary models are GPT-5-mini, GPT-5.1, GPT-5.4-mini, GPT-5.5 [28], Claude Sonnet 4.6 [3], and Claude Opus 4.7 [2]. The open-source models are Qwen3-VL-8B, Qwen3-VL-30B [4], Qwen3-8B [40], DeepSeek-Chat, and DeepSeek-Reasoner [18]. All experiments are done with 12 NVIDIA A6000 Ada GPUs and an AMD EPYC 7313 CPU.

EDA Tools and Semiconductor Nodes. The primary goal of our work is to benchmark the capabilities of LLMs/VLMs for VLSI physical design, not to compare or rank different EDA tools. Commercial EDA tools are used only as part of the evaluation infrastructure to assess the designs guided by LLMs/VLMs. We use a privacy-preserving framework to integrate commercial EDA tools and LLMs/VLMs without exposing the initial scripts (Section A.9). We use Cadence Innovus 22 [5], Synopsys IC Compiler II (ICC2) 2022 [30], Synopsys PrimeTime (PT) 2022 [31], and Synopsys Formal (FM) 2022 [29] as our commercial EDA tools for implementation and signoff. Command-level differences between tool versions are minimal; we have validated that our generated scripts are compatible with versions 2018 and 2020. For experiments using commercial EDA tools, we target the TSMC 28-nm process; for the open-source flow, we adopt OpenROAD with the Nangate 45-nm node.

Evaluation Metrics and Sampling Configuration. We report **pass@1** and **pass@5** for *Physical Design Understanding* and *Script Generation* tasks. For free-response foundational knowledge, root-cause analysis, and report comprehension tasks, we generate five responses per query and score each against the reference answer using predefined rubrics. Pass@1 is obtained from a single response generated with temperature 0 (deterministic decoding). Pass@5 is computed by generating five responses with temperature 0.8 (top-p = 0.95) and reporting the best rubric score among them. For full-flow implementation tasks, we report **stage-specific design metrics**, including WNS, placement density, and the number of DRC violations, which directly reflect design quality and PPA outcomes. For open-source models, we use deterministic decoding (temperature = 0) for pass@1, and stochastic sampling (temperature = 0.8, top-p = 0.95) for pass@5. For proprietary APIs that do not expose a temperature parameter, we sample five responses per query using the default configuration.

LLM-as-Judge and Human Evaluation. To enable affordable, large-scale qualitative analysis of model outputs, we adopt the widely used LLM-as-Judge protocol [43] with Claude Sonnet 4.6 as the judge. Full prompts are provided in Appendix A.7. To validate our LLM-Judge protocol, we recruit three senior physical-design engineers to independently score the model outputs on Basic, RootOpt, and Report using the same rubric provided to the LLM judge. We report the mean score across the three engineers as the human-evaluation result in Table 4.

4.2 Experimental Results

Physical Design Understanding and Script Generation. Table 4 summarizes the key results, with per-task pass@1 of the top four models illustrated in Figure 3. GPT-5.5 achieves state-of-the-art performance on five of eight tasks, attaining 45.5% and 42.2% pass@1 on ICC2 and Innovus script generation, respectively, representing up to a 4× improvement over the strongest open-source model (Qwen3-VL-30B, at 9.1% and 11.1%). Despite this progress, all models remain below 50% pass@1 on script generation and 60% on ECO tasks, highlighting the difficulty of producing executable, tool-specific Tcl code and the limited coverage of EDA-specific data in public training corpora. In contrast, foundational knowledge emerges as the most tractable category: Claude Opus 4.7 achieves the highest performance at 82.4%, while even open-source models reach competitive levels (e.g., Qwen3-VL-30B at 69.2%), suggesting that textbook-level physical design knowledge is well represented in pretraining data. For reasoning-intensive tasks, including root-cause analysis and report comprehension, GPT-5.5 exceeds 70% pass@1, whereas open-source models lag by 20~40 percentage points. This substantial gap indicates that complex reasoning and multi-step diagnosis are key differentiators between proprietary and open models in physical design settings. Human scores (parentheses in Table 4) track LLM-Judge scores within 2~6 points, demonstrating that LLM-as-Judge scoring is a reliable proxy for human evaluation on these tasks.

Full-Flow Implementation. Table 5 reports partial full-flow implementation results of PDAGENT on three representative designs from the PDAGENT-Bench full-flow suite (i.e., TinyRISCV, AES-256, and Ethernet MAC); complete results and an ablation against an LLM/VLM agent without our skill library are provided in the Appendix A.3. The skill-free baseline fails at early stages such as initialization and floorplanning, consistent with the weak script-generation performance reported in Table 4. In contrast, PDAGENT closes timing (positive WNS) and resolves all DRC violations after RouteOpt across the three designs, demonstrating that our agent can drive a complete RTL-to-GDSII flow on real industrial designs in TSMC 28 nm.

Macro Placement. The macro placement is the grand challenge in the VLSI physical design workflow, which still lacks effective algorithms. We thus evaluate VLM agents enhanced by Skills in

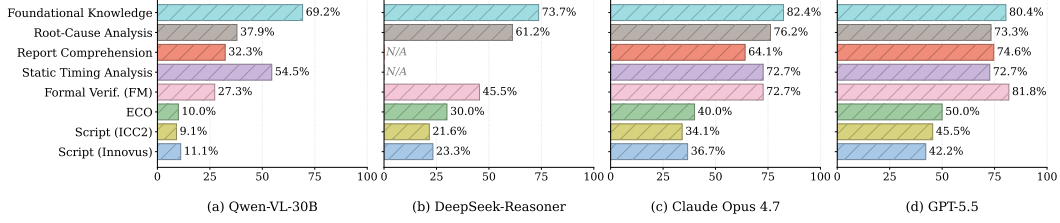


Figure 3: Per-task accuracy (%) of four LLMs across eight evaluation categories. Frontier models (Claude Opus 4.7, GPT-5.5) lead across nearly all tasks, while all models struggle on script generation and ECO. N/A denotes tasks requiring multi-modal input, which the model (LLM) does not support.

Table 4: Benchmark results across Physical Design Understanding and Script Generation. Each cell reports LLM-Judge pass@1 / pass@5; values in parentheses are the corresponding *human-evaluation* scores, available for Foundational knowledge (Foundational), RootOpt, and Report. **Bold** indicates the best LLM-Judge result per column; underline indicates the second best. N/A: model does not support multi-modal input.

Model	Foundational		RootOpt		Report		STA		FM		ECO		P&R Vendor 1		P&R Vendor 2	
	p@1	p@5	p@1	p@5	p@1	p@5	p@1	p@5	p@1	p@5	p@1	p@5	p@1	p@5	p@1	p@5
Qwen-VL-8B	.533 (.520)	.573 (.558)	.245 (.228)	.319 (.269)	.327 (.293)	.464 (.421)	.545	<u>.636</u>	.273	.455	.100	.100	.068	.102	.122	.133
Qwen-VL-30B	.692 (.661)	.695 (.679)	.379 (.368)	.460 (.400)	.323 (.294)	.427 (.415)	.545	<u>.636</u>	.273	.545	.100	.100	.091	.125	.111	.178
Qwen3-8B	.543 (.488)	.585 (.573)	.255 (.216)	.286 (.263)	N/A	N/A	N/A	.273	.500	.000	.000	.080	.102	.122	.144	
DeepSeek-Chat	<u>.726</u> (.689)	<u>.799</u> (.773)	.486 (.434)	.521 (.473)	N/A	N/A	N/A	.409	.591	.300	<u>.500</u>	.340	.364	.256	.356	
DeepSeek-Reasoner	<u>.737</u> (.684)	<u>.806</u> (.788)	.612 (.585)	.680 (.630)	N/A	N/A	N/A	.455	.591	.300	<u>.500</u>	.216	.352	.233	.356	
GPT-5-mini	.800 (.701)	<u>.820</u> (.790)	.560 (.540)	.655 (.615)	.655 (.607)	.685 (.666)	.545	<u>.636</u>	.227	.364	.300	.300	.057	.114	.133	.189
GPT-5.1	.820 (.802)	<u>.840</u> (.830)	.655 (.619)	.674 (.624)	.655 (.604)	.705 (.649)	.545	.545	.545	.636	.200	.200	.318	<u>.432</u>	.278	.311
Claude Sonnet 4.6	.826 (.794)	.835 (.821)	.688 (.660)	.700 (.664)	<u>.668</u> (.625)	<u>.736</u> (.717)	<u>.636</u>	<u>.636</u>	.636	.682	.500	<u>.500</u>	<u>.341</u>	.352	.289	.311
Claude Opus 4.7	<u>.824</u> (.784)	.842 (.829)	.762 (.714)	<u>.767</u> (.731)	.641 (.614)	.659 (.629)	.727	.727	<u>.727</u>	<u>.727</u>	<u>.400</u>	<u>.500</u>	<u>.341</u>	.375	<u>.367</u>	<u>.411</u>
GPT-5.5	.804 (.778)	.830 (.807)	<u>.733</u> (.689)	.774 (.732)	.746 (.720)	.750 (.732)	.727	.727	.818	.864	.500	.600	.455	.580	.422	.511
GPT-5.4-mini	.723 (.668)	.741 (.726)	.545 (.502)	.581 (.526)	.461 (.422)	.514 (.487)	<u>.636</u>	<u>.636</u>	.591	<u>.727</u>	.500	<u>.500</u>	.295	.398	.267	.356

Table 5: Partial results of **PDAgent** on the PDAgent-Bench Full flow suite (TSMC 28nm); the complete results are reported in Appendix A.3. WNS and DRC violations are reported at each stage; positive WNS indicates timing closure. Utilization (Util.) is reported after floorplanning.

Design	Init	Floorplan	Powerplan	Place	CTS	CTSopt	Route	Routeopt
TinyRISCV	5ns, WNS: 1.362ns	Util: 0.65	DRC Vio: 0	WNS : -0.014ns	WNS : -0.127ns	WNS : 0.001ns	WNS : 0.021ns	WNS : 0.021ns
	Setup successfully	DRC Vio: 0		DRC Vio: 0	DRC Vio: 0	DRC Vio: 0	DRC Vio: 1	DRC Vio: 0
AES-256	3.5ns, WNS: 1.003ns	Util: 0.60	DRC Vio: 0	Density: 0.715	Density: 0.7165	Density: 0.713	Density: 0.722	Density: 0.722
	Setup successfully	DRC Vio: 0		WNS : 0.003ns	WNS : 0.006ns	WNS : 0.006ns	WNS : 0.12ns	WNS : 0.119ns
Ethernet MAC	10ns, WNS: 5.621ns	Util: 0.60	DRC Vio: 0	DRC Vio: 0	DRC Vio: 0	DRC Vio: 0	DRC Vio: 303	DRC Vio: 0
	Setup successfully	DRC Vio: 0		Density: 0.613	Density: 0.615	Density: 0.615	Density: 0.615	Density: 0.615
				WNS : 5.62ns	WNS : 5.475ns	WNS : 5.470ns	WNS : 5.53ns	WNS : 5.53ns
				DRC Vio: 25	DRC Vio: 23	DRC Vio: 0	DRC Vio: 69	DRC Vio: 0
				Density: 0.62	Density: 0.624	Density: 0.628	Density: 0.624	Density: 0.625

our framework on this specific design stage. We evaluate them on five ChiPBench designs against seven learning-based and analytical placers: WireMask-EA, simulated annealing (SA), MaskPlace, ChiPFormer, DREAMPlace, AutoDMP, and OpenROAD. The detailed results are reported in Table 12 (Appendix A.5), demonstrating that our flow, driven by an 8B VLM backbone, can effectively handle complex macro placement and match or outperform specialized macro placers in timing closure.

4.3 Key Findings and Insights

Key Findings. We analyze the quantitative results alongside error cases (Appendix A.2) and summarize the findings, revealing the main challenges and directions for improving LLM-based agentic workflows in physical design. **Finding 1: Failures are predominantly syntactic rather than conceptual.** Approximately 92% of script-generation failures are near-misses, where models select the correct command (e.g., in Cadence Innovus) but produce incorrect, missing, or inconsistent arguments. This suggests that LLMs largely capture the *procedural structure* of physical design workflows, but lack fidelity in tool-specific syntax. Bridging this gap requires explicit **syntax grounding** or retrieval over tool documentation. **Finding 2: Strong performance on foundational knowledge.** On the foundational benchmark, GPT-5.5 achieves 80.4% overall across 90 questions, including 83.0% on easy and 81.7% on medium difficulty. This indicates that frontier models can

reliably answer conceptual and entry-level physical design questions, serving as a strong baseline for knowledge understanding. **Finding 3: Perception outpaces decision-making.** While models achieve near-perfect performance on report comprehension ($\sim 9/10$), performance drops significantly ($\sim 5/10$) when tasks require root-cause diagnosis or optimization actions. This reveals a gap between **interpreting tool outputs** and **acting on them**, highlighting the need for mechanisms that translate observations into executable design decisions. **Finding 4: Skill augmentation enables end-to-end execution.** Integrating curated skill libraries substantially improves performance on full-flow tasks (including the challenging macro placement task) by reducing reliance on fragile script generation. With access to reusable procedures, agents focus on higher-level reasoning, e.g., navigating PPA trade-offs and iteratively refining designs, enabling more stable closed-loop execution.

Insights. These findings suggest two complementary directions for advancing LLM/VLM-based physical design. (i) *Data and training.* Expanding datasets that align reports with root-cause analyses, optimization rationales, and executable scripts can directly address gaps in syntax fidelity and decision-making (**Findings 1 and 3**). (ii) *Skill-centric architectures.* Developing richer and more scalable skill libraries, along with efficient retrieval and composition mechanisms, can improve generalization across designs, tools, and technology nodes (**Finding 4**). Finally, training domain-adapted models on such curated data would enable **on-premises deployment**, which is essential for industrial physical design workflows involving proprietary designs and process design kits (PDKs).

4.4 Limitations and Future Work

Limitations. While PDAGENT-BENCH provides a comprehensive framework for evaluating LLM/VLMs agents in physical design, several limitations remain. First, **signoff completeness** is not fully covered. Layout-versus-Schematic (LVS) verification is currently excluded, leaving a critical stage of the design flow unmodeled. Second, **automation coverage** is incomplete for certain tasks, particularly DRC fixing that relies on GUI-based interactions or manual inspection, which are difficult to capture within script-driven workflows. Third, **dataset scale and diversity** are limited: although the benchmark includes both foundational and production-oriented tasks, it does not yet fully reflect the scale, heterogeneity, and corner cases of industrial-grade designs.

Future Work. We outline several directions to extend both the benchmark and the agentic design framework. **First**, we plan to expand **technology coverage** beyond 28-nm to include both advanced nodes (e.g., 16-nm, 7-nm) and mature nodes (e.g., 65-nm, 130-nm), enabling broader evaluation across design regimes. **Second**, we will increase **dataset scale and complexity** by incorporating larger and more diverse designs, improving the benchmark’s ability to stress-test long-horizon reasoning and workflow robustness. **Third**, we aim to develop more **fine-grained, stage-specific capabilities**, including automated macro placement, congestion-aware floorplanning, and enhanced DRC/LVS handling, to better approximate real-world design workflows. **Fourth**, we will **enhance the general capabilities** of our agentic platform by incorporating modules currently absent from the framework, such as long-term memory and self-improvement mechanisms that enable the agent to learn from past successes and failures. **Finally**, we plan to improve **evaluation fidelity** by incorporating additional signoff metrics and richer feedback signals, further bridging the gap between benchmark settings and production environments.

5 Conclusion

In this work, we introduce PDAGENT-BENCH, the first comprehensive benchmark for systematic evaluation of LLM/VLMs agents in VLSI physical design, comprising 353 tasks across five capability axes and a unified multi-agent framework for closed-loop interaction with EDA tools. Our evaluation of 11 frontier LLMs/VLMs reveals a substantial gap between conceptual understanding and tool-grounded execution. We anticipate PDAGENT-BENCH will accelerate the development of domain-specific LLM/VLMs agents for this underexplored but high-impact domain.

References

- [1] A. Abdelazeem. Systolic array implementation in RTL for TPU. <https://github.com/abdelazeem201/Systolic-array-implementation-in-RTL-for-TPU>, 2021. Accessed: Apr. 2026.

- [2] Anthropic. Introducing claude opus 4.7, 2026. URL <https://www.anthropic.com/news/claude-opus-4-7>.
- [3] Anthropic. Introducing claude sonnet 4.6, 2026. URL <https://www.anthropic.com/news/claude-sonnet-4-6>.
- [4] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [5] Cadence. Innovus implementation system. URL https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/soc-implementation-and-floorplanning/innovus-implementation-system.html.
- [6] C. Chen, X. Xiang, C. Liu, Y. Shang, R. Guo, D. Liu, Y. Lu, Z. Hao, J. Luo, Z. Chen, et al. Xuantie-910: A commercial multi-core 12-stage pipeline out-of-order 64-bit high performance risc-v processor with vector extension: Industrial product. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 52–64. IEEE, 2020.
- [7] Q. Cheng, L. Lin, M. Huang, Q. Li, Z. Yang, L. Dai, H. Yu, Y.-J. Chen, Y. Shi, and M. Hashimoto. A 13-34 tops/w edge-ai processor featuring booth-value-confined accelerator, near-memory computing, and contiguity-aware mapping. In *2024 IEEE Asian Solid-State Circuits Conference (A-SSCC)*, pages 1–3, 2024. doi: 10.1109/A-SSCC60305.2024.10849341.
- [8] Q. Cheng, Q. Li, W. Dong, M. Zhang, R. Zhang, M. Huang, H. Yu, Y. Shi, H. Awano, T. Sato, L. Lin, and M. Hashimoto. A 22nm resource-frugal hyper-heterogeneous multi-modal system-on-chip towards in-orbit computing. In *2025 IEEE Custom Integrated Circuits Conference (CICC)*, pages 1–3, 2025. doi: 10.1109/CICC63670.2025.10983627.
- [9] Q. Cheng, Q. Li, Z. Yang, Z. Kong, G. Niu, Y. Liang, J. Li, J. H. Park, W. Liao, H. Awano, T. Sato, L. Lin, and M. Hashimoto. A radiation-hardened neuromorphic imager with self-healing spiking pixels and unified spiking neural network for space robotics. In *2025 Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, pages 1–3, 2025. doi: 10.23919/VLSITechnologyandCir65189.2025.11075180.
- [10] Q. Cheng, Z. Yang, H. Li, Q. Li, Z. Kong, G. Niu, Y. Liang, J. Li, J. Yoo, M. Hashimoto, and L. Lin. A radiation-hardened self-healing cmos imager with online pixel/logic annealing and tile-adaptive compression for space applications. In *2026 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 69, pages 390–392, 2026. doi: 10.1109/ISSCC49663.2026.11408995.
- [11] A. Ghose, A. B. Kahng, S. Kundu, Y. Liu, B. Pramanik, Z. Wang, and D. Yoon. Use cases and deployment of ml in ic physical design. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference, ASPDAC '25*, page 669–675, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400706356. doi: 10.1145/3658617.3703143. URL <https://doi.org/10.1145/3658617.3703143>.
- [12] A. Ghose, A. B. Kahng, S. Kundu, and Z. Wang. Orfs-agent: Tool-using agents for chip design optimization. In *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*, pages 1–13, 2025. doi: 10.1109/MLCAD65511.2025.11189204.
- [13] C.-T. Ho, H. Ren, and B. Khailany. Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 300–307, 2025.
- [14] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu. *VLSI physical design: from graph partitioning to timing closure*, volume 312. Springer, 2011.
- [15] J. Kindér. AES: Verilog implementation of the advanced encryption standard (AES-256). <https://github.com/secworks/aes>, 2014. Accessed: Apr. 2026.
- [16] L. Lavagno, L. Scheffer, and G. Martin. *EDA for IC implementation, circuit design, and process technology*. CRC press, 2018.

- [17] K. Liang. TinyRISC-V: A simple RISC-V core. <https://github.com/liangkangnan/tinyriscv>, 2020. Accessed: Apr. 2026.
- [18] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [19] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu, et al. Chipnemo: Domain-adapted llms for chip design. *arXiv preprint arXiv:2311.00176*, 2023.
- [20] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie. Openllm-rtl: Open dataset and benchmark for llm-aided design rtl generation, 2025. URL <https://arxiv.org/abs/2503.15112>.
- [21] W.-H. Liu, S. Mantik, W.-K. Chow, Y. Ding, A. Farshidi, and G. Posser. Ispd 2019 initial detailed routing contest and benchmark with advanced routing rules. In *Proceedings of the 2019 international symposium on physical design*, pages 147–151, 2019.
- [22] Y. Lu, S. Liu, Q. Zhang, and Z. Xie. Rtlrm: An open-source benchmark for design rtl generation with large language model, 2023. URL <https://arxiv.org/abs/2308.05345>.
- [23] NVIDIA. NVDLA: Open source deep learning accelerator. <https://github.com/nvdla/hw>, 2017. Accessed: Apr. 2026.
- [24] OpenCores. Ethernet MAC 10/100 Mbps. <https://opencores.org/projects/>, 2001. Accessed: Apr. 2026.
- [25] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany. Revisiting verilogeval: Newer llms, in-context learning, and specification-to-rtl tasks. *arXiv preprint arXiv:2408.11053*, 2024.
- [26] P. Shrestha, A. Aversa, S. Phatharodom, and I. Savidis. Eda-schema: A graph datamodel schema and open dataset for digital design automation. In *Proceedings of the Great Lakes Symposium on VLSI 2024, GLSVLSI '24*, page 69–77, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706059. doi: 10.1145/3649476.3658718. URL <https://doi.org/10.1145/3649476.3658718>.
- [27] P. Shrestha, A. Aversa, and I. Savidis. Eda-schema-v2: A multimodal schema, open datasets, and benchmarks for machine learning in digital physical design, 2026. URL <https://arxiv.org/abs/2605.06952>.
- [28] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, A. Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- [29] Synopsys. Vc formal: Formal verification solution, . URL <https://www.synopsys.com/verification/static-and-formal-verification/vc-formal.html>.
- [30] Synopsys. Ic compiler ii: Place & route solution, . URL <https://www.synopsys.com/implementation-and-signoff/physical-implementation/ic-compiler.html>.
- [31] Synopsys. Primetime static timing analysis, . URL <https://www.synopsys.com/implementation-and-signoff/signoff/primetime.html>.
- [32] Ultra-Embedded. UART controller IP core. <https://github.com/ultraembedded/cores>, 2020. Accessed: Apr. 2026.
- [33] M. Wang, Y. Wen, Y. Lu, F. Liu, Y. Zhao, B. Han, J. Mu, Y. Lin, R. Wang, B. Yu, et al. Circuitnet 3.0: A multi-modal dataset with task-oriented augmentation for ai-driven circuit design. In *The Fourteenth International Conference on Learning Representations*.
- [34] Z. Wang, Z. Geng, Z. Tu, J. Wang, Y. Qian, Z. Xu, Z. Liu, S. Xu, Z. Tang, S. Kai, et al. Benchmarking end-to-end performance of ai-based chip placement algorithms. *arXiv preprint arXiv:2407.15026*, 2024.
- [35] N. H. Weste and D. Harris. *CMOS VLSI design: a circuits and systems perspective*. Pearson Education India, 2015.

- [36] C. S. D. Workshop. Toward cognitive silicon design: Structural barriers and an industry roadmap for ai-native chip design. URL `chrome-extension://efaidnbmnnnibpcajpcgiclfindmkaj/https://vlsicad.ucsd.edu/NEWS26_2/TowardsCognitiveSiliconDesign-04032026_Workshop_Final.pdf`.
- [37] B.-Y. Wu, U. Sharma, A. Rovinski, and V. A. Chhabria. Openroad agent: An intelligent self-correcting script generator for openroad. In *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*, pages 16–22, 2025. doi: 10.1109/ICLAD65226.2025.00039.
- [38] H. Wu, Z. He, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu. Chateda: A large language model powered autonomous agent for eda. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(10):3184–3197, 2024.
- [39] N. Xu, Z. Zhang, S. Shu, L. Qi, J. Lv, W. Wang, T. Zhao, C. Zhang, Z. Yang, X. Li, et al. iscript: A domain-adapted large language model and benchmark for physical design tcl script generation. *arXiv preprint arXiv:2603.04476*, 2026.
- [40] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [41] B. Yu. Machine learning in eda: When and how. In *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, pages 1–6. IEEE, 2023.
- [42] G. Zhang, H. Geng, X. Yu, Z. Yin, Z. Zhang, Z. Tan, H. Zhou, Z. Li, X. Xue, Y. Li, et al. The landscape of agentic reinforcement learning for llms: A survey. *arXiv preprint arXiv:2509.02547*, 2025.
- [43] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.
- [44] Y. Zhu, D. Huang, H. Lyu, X. Zhang, C. Li, W. Shi, Y. Wu, J. Mu, J. Wang, Y. Zhao, et al. Qimeng-codev-r1: Reasoning-enhanced verilog generation. *arXiv preprint arXiv:2505.24183*, 2025.

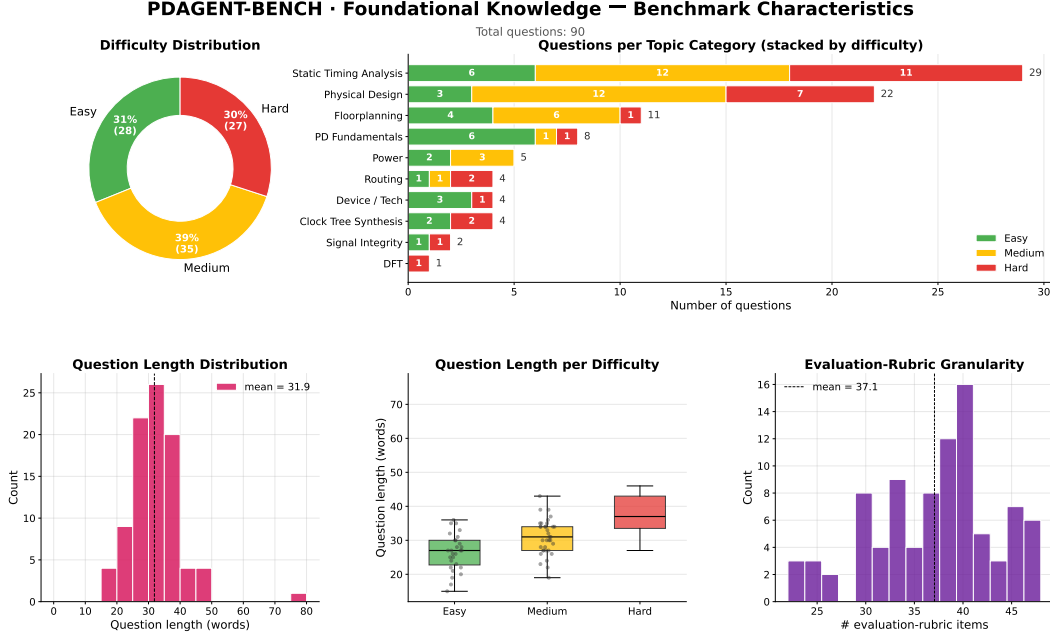


Figure 4: Characteristics of the foundational-knowledge subset of PDAGENT-BENCH (90 questions). Questions are balanced across three difficulty levels and span ten physical-design topics, with timing analysis and physical design as the most represented categories. Each question is paired with a fine-grained evaluation rubric (mean of 37.1 items).

A Appendix

A.1 Characteristics of PDAGENT-BENCH

We summarize the key characteristics of each subset in Figures 4–7, including question count, difficulty distribution, topic coverage, and evaluation-rubric granularity. Together, the four subsets comprise 343 questions spanning the full physical-design workflow, from foundational concepts to tool-specific script generation.

A.2 Error Case Analysis

To understand the failure modes of current LLMs on physical-design tasks, we manually inspect the incorrect outputs of all evaluated models, categorize them by error type, and report the resulting distribution. The analysis surfaces the systematic weaknesses underlying the quantitative gaps reported in Table 4 and informs the directions discussed in Section 4.3.

Table 6: Failure breakdown of ChatGPT-5.5 on `benchmark_innovus.json` (52 failures out of 90).

Category	Count	Description
Multi-line near-miss	33	Right intent/commands, wrong flag names or missing/extra steps
Same command, wrong attribute	12	Single-line command correct, but flag is wrong
Same command, value formatting	3	Right command + flags, formatting/value diff
Wrong command	4	Picked wrong command (incl. 1 free-text answer)
Total	52	

A.3 Results on Full Flow Implementation

The evaluation results of **PDAGENT** on the PDAGENT-Bench holistic flow suite are summarized in Table 9, using GPT-5mini as the backbone model. The results demonstrate that **PDAGENT** is capable of executing a holistic physical-design flow end-to-end, performing self-iterative design space exploration, autonomously resolving DRC violations, and optimizing the design toward timing

PDAGENT-BENCH · RootOpti (Root-Cause) — Benchmark Characteristics

Total questions: 21

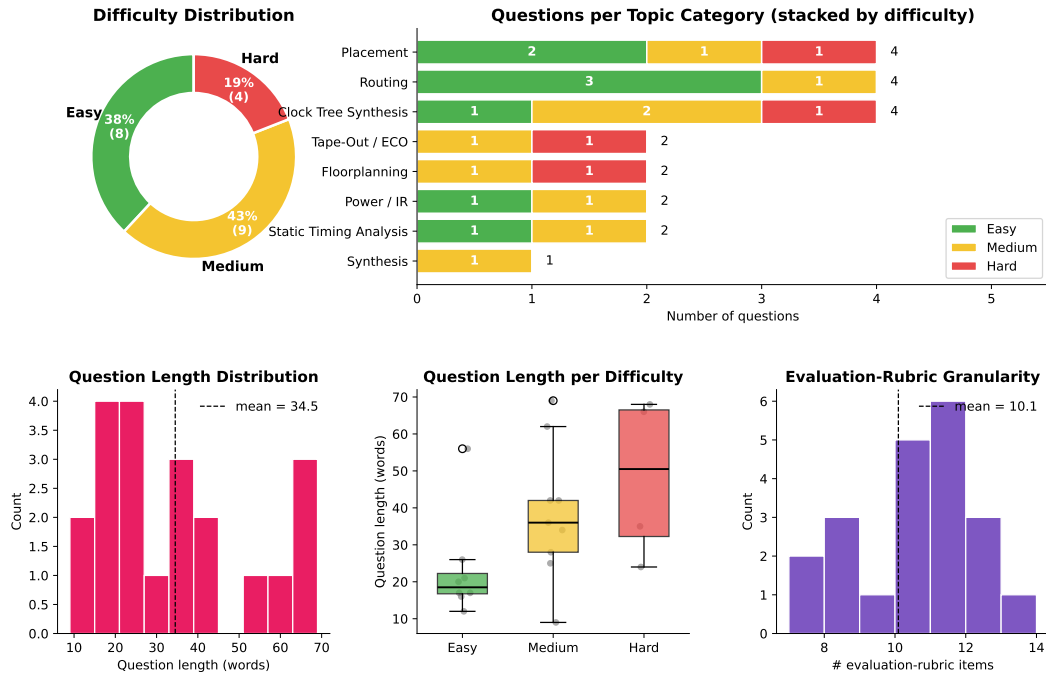


Figure 5: Characteristics of the root-cause analysis subset of PDAGENT-BENCH (21 questions). Questions span eight physical-design stages, from synthesis through tape-out/ECO, with placement, routing, and clock tree synthesis as the most represented categories. Each question is paired with a targeted evaluation rubric (mean of 10.1 items).

PDAGENT-BENCH · Report Comprehension — Benchmark Characteristics

Total questions: 11

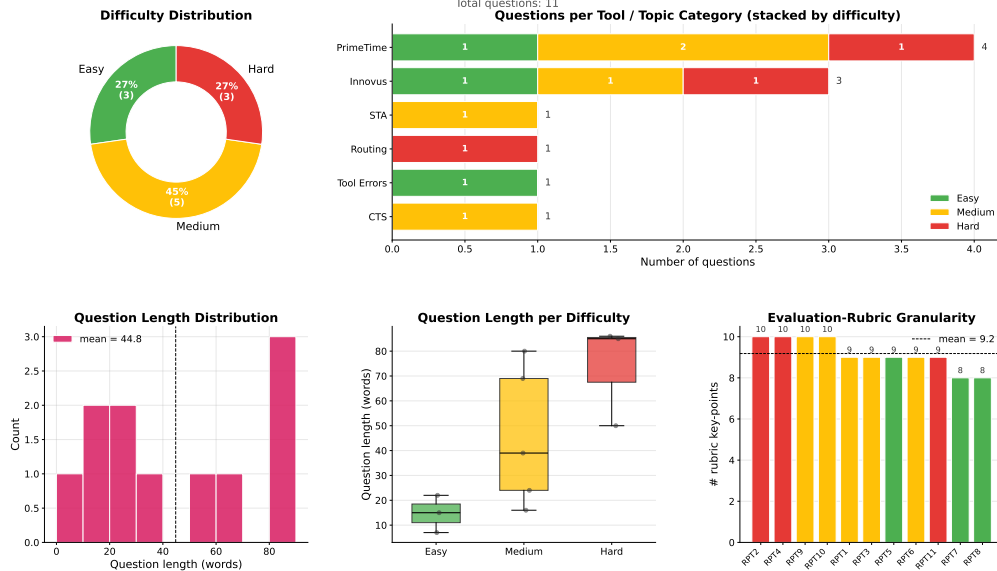


Figure 6: Characteristics of the report-comprehension subset of PDAGENT-BENCH (11 questions). Questions cover reports from major EDA tools (PrimeTime, Innovus) across stages including STA, routing, CTS, and tool error logs. Each report is paired with a structured evaluation rubric.

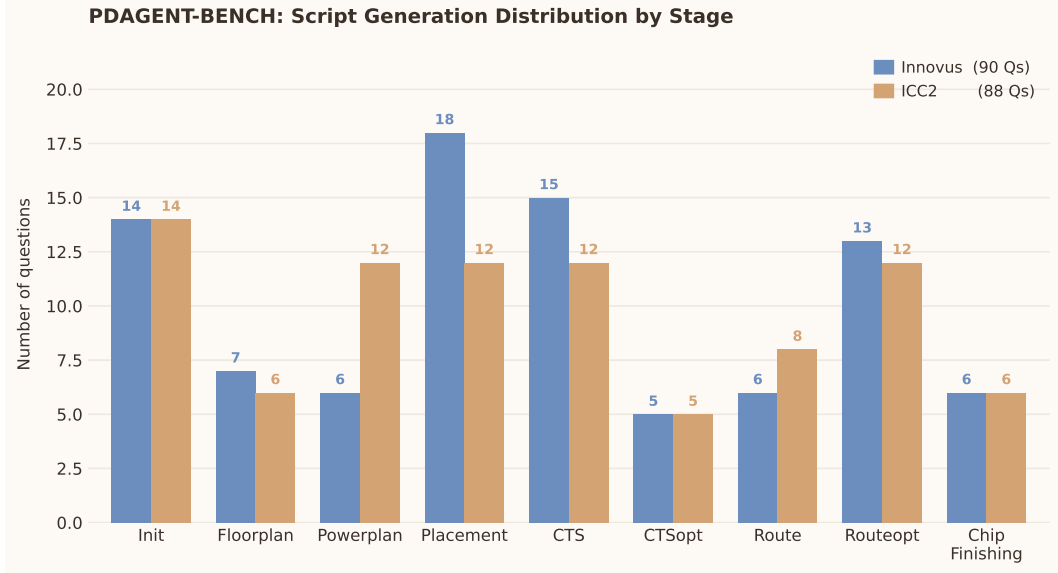


Figure 7: Characteristics of the script generation subset of PDAGENT-BENCH (178 questions in total). Questions span nine physical-design stages, from initialization through chip finishing, and are provided in parallel for two industrial EDA tools, Innovus (90 questions) and ICC2 (88 questions). Placement, CTS, and routing-optimization stages account for the largest share of questions in both tools.

Table 7: Examples of attribute-name hallucinations: the model picks the correct command but invents plausible-but-wrong Innovus flag names.

ID	Expected flag	Generated flag
TC_016	-around core -center 1	-follow core
TC_018	-subclass	-userDefined
TC_021	-no_routing_blk	-ignore_routing_blockage
TC_026	-rule ... -wellCutCell	-distance ... -boundaryCell
TC_038	-place_opt_save_db	-place_global_save_db
TC_054	-outDir	-report_dir
TC_069	-drouteMinSlackForWireOptimization	-droutePostRouteSpreadWireMinSlack
TC_077	lefDefOutVersion	defOutVersion
TC_084	specifyCellPad CLK* (positional)	specifyCellPad -cell CLK*

closure. The consistent performance across diverse designs demonstrates the generality of our method. On average, **PDAgent** completes the full flow through 53 interactions with the EDA tools, reflecting the iterative nature of physical-design optimization.

Our method performs well on small and medium-complexity designs, achieving timing closure with zero or minimal DRC violations at the final RouteOpt stage. However, on larger and more complex designs such as NVDLA-large and Open910, residual DRC violations remain after RouteOpt, and timing closure is not fully achieved in all cases. We attribute these limitations not only to the inherent complexity of the design space at advanced technology nodes, but also to the current lack of more sophisticated ECO and DRC-fixing capabilities in our framework, which we identify as a promising direction for future work. We anticipate that future work building upon **PDAgent** can achieve improved timing slack, higher utilization, cleaner DRC results, and reduced interaction overhead through more advanced optimization strategies.

Comparison Against RAG-enhanced Agent Baseline

We further evaluate two RAG-enhanced LLM baselines, which retrieve relevant content from EDA command reference manuals and physical-design guidebooks, using GPT-5mini backbone models for fair comparison. The Full flow suite results are reported in Table 10.

Table 8: GPT-5.5 Basic Benchmark Executive Summary (90 questions).

Overall Score: 724.0 / 900.5 = 80.4%	
By Difficulty	Easy 83.0% Medium 81.7% Hard 76.0%
Catastrophic Fails (<60%)	2 / 90 (→ Q2: 15%, Q86: 45%)
Low Criteria (≤50% of max)	32 / 454 (~7% — concentrated, not diffuse)
Strengths vs. Weaknesses	
Strengths	• File formats (GDSII / SDF / DEF, all ≥85%) • STA definitions (slack, false path, best/worst path: 100%) • Technology-node comparison (Q63: 100%) • Digital logic fundamentals • Physical Design Fundamentals topic family (89%)
Weaknesses	• Question-interpretation errors (Q2) • Quantitative reasoning / layer math (Q86) • Library models — NLDM vs. CCS (65%) • CTS insertion-delay tuning • Post-route to signoff correlation
Top 3 Error Patterns	
1	Misreading the question’s framing → wrong taxonomy (Q2).
2	Quantitative reasoning with constraint propagation (Q86).
3	“Missing one column” — partial trade-off tables, lost bonus points.
Verdict	
Strong general PD/STA knowledge; points are lost to under-elaboration and one-off interpretation errors, not deep knowledge gaps.	

For the Full flow suite, the RAG-enhanced method fails to complete most designs beyond the first two stages. We attribute this performance gap to the highly structured and interdependent nature of physical-design flows: each stage comprises multiple steps with precise tool-level syntax and context-specific requirements, which general-purpose retrieved documentation is too coarse-grained to fully capture. Unlike **PDAgent**, which encodes actionable, stage-specific skills, the RAG-based approach lacks the procedural knowledge necessary to handle tool-specific errors, adapt parameters across designs, and drive the flow to completion.

A.4 Token Usage of full flow implementation

Table 11 reports token consumption for the full flow across the ten benchmarks. The cost is overwhelmingly dominated by input (context) tokens: 5.84M. This is expected for an agentic EDA loop, in which each step ingests large synthesis logs, timing reports, and netlist summaries but emits only short commands and decisions. Token usage tracks design complexity: NVDLA-large, the largest design (1.48M cells), is the most expensive at 1.62M tokens and 192 calls, whereas the small control designs (UART, Elevator FSM, 16b multiplier) converge in 27–34 calls and under 180k tokens each.

A.5 Macro Placement results

We evaluate on five ChiPBench designs against seven learning-based and analytical placers (WireMask-EA, SA, MaskPlace, ChiPFormer, DREAMPlace, AutoDMP, and OpenROAD) to show that our platform can handle the complex macro placement. For each design, the model generates five candidate placements that are refined in parallel, with each candidate given up to five rounds of feedback-driven refinement. As shown in Table 12, our flow achieves the best WNS and TNS on all four multi-baseline designs, cutting TNS by up to ~60× over the strongest baseline (e.g., −244.94 → −4.01 on ariane133) and staying on par on swerv_wrapper. It also attains the lowest

Table 9: Full-flow implementation results of **PDAgent** on the PDAgent-Bench full flow suite (TSMC 28nm). WNS and DRC violations are reported at each stage; positive WNS indicates timing closure. Utilization (Util.) is reported after floorplanning.

Design	init	floorplan	powerplan	place	CTS	CTSopt	Route	Routeopt
TinyRISCV	5ns, WNS: 1.362ns	Util: 0.65	DRC Vio: 0	WNS : -0.014ns	WNS : -0.127ns	WNS : 0.001ns	WNS : 0.021ns	WNS : 0.021ns
	Setup successfully	DRC Vio: 0		DRC Vio: 0	DRC Vio: 0	DRC Vio: 0	DRC Vio: 1	DRC Vio: 0
Open910	1.5ns, WNS: 0.465ns	Util: 0.60	DRC Vio: 0	Density: 0.715	Density: 0.7165	Density: 0.713	Density: 0.722	Density: 0.722
	Setup successfully	DRC Vio: 0		WNS : -0.05ns	WNS : -0.009ns	WNS : 0.001ns	WNS : -0.431ns	WNS : 0.008ns
Systolic array	3.4ns, WNS: 1.11ns	Util: 0.5	DRC Vio: 0	DRC Vio: 55	DRC Vio: 2	DRC Vio: 0	DRC Vio: 127	DRC Vio: 125
	Setup successfully	DRC Vio: 0		Density: 0.584	Density: 0.653	Density: 0.654	Density: 0.703	Density: 0.731
16b pipelined mult.	2.6ns, WNS: 0.808ns	Util: 0.55	DRC Vio: 0	WNS : 0.013ns	WNS : 0.003ns	WNS : 0.003ns	WNS : -0.312ns	WNS : 0.002ns
	Setup successfully	DRC Vio: 0		DRC Vio: 0	DRC Vio: 0	DRC Vio: 0	DRC Vio: 0	DRC Vio: 0
AES-256	3.5ns, WNS: 1.003ns	Util: 0.60	DRC Vio: 0	Density: 0.464	Density: 0.464	Density: 0.464	Density: 0.464	Density: 0.466
	Setup successfully	DRC Vio: 0		WNS : 0.02ns	WNS : -0.008ns	WNS : 0.012ns	WNS : 0.251ns	WNS : 0.251ns
NVDLA-small	2ns, WNS: 0.65ns	Util: 0.60	DRC Vio: 0	DRC Vio: 0	Density: 0.6289	Density: 0.629	Density: 0.629	Density: 0.629
	Setup successfully	DRC Vio: 0		WNS : 0.003ns	WNS : 0.006ns	WNS : 0.006ns	WNS : 0.12ns	WNS : 0.119ns
NVDLA-large	2.5ns, WNS: 0.69ns	Util: 0.60	DRC Vio: 0	Density: 0.613	Density: 0.615	Density: 0.615	Density: 0.615	Density: 0.615
	Setup successfully	DRC Vio: 0		WNS : -1.06ns	WNS : 0.002ns	WNS : 0.002ns	WNS : -0.005ns	WNS : 0.002ns
UART controller	3.4ns, WNS: 1.108ns	Util: 0.65	DRC Vio: 0	DRC Vio: 46	DRC Vio: 0	DRC Vio: 0	DRC Vio: 21	DRC Vio: 19
	Setup successfully	DRC Vio: 0		Density: 0.424	Density: 0.554	Density: 0.554	Density: 0.556	Density: 0.556
Elevator FSM	2.5ns, WNS: 0.708ns	Util: 0.7	DRC Vio: 0	WNS : -0.024ns	WNS : -0.145ns	WNS : -0.063ns	WNS : 0.021ns	WNS : 0.021ns
	Setup successfully	DRC Vio: 0		DRC Vio: 68	DRC Vio: 0	DRC Vio: 0	DRC Vio: 85	DRC Vio: 85
Ethernet MAC	10ns, WNS: 5.621ns	Util: 0.60	DRC Vio: 0	Density: 0.487	Density: 0.586	Density: 0.594	Density: 0.596	Density: 0.598
	Setup successfully	DRC Vio: 0		WNS : 1.1ns	WNS : 1.4ns	WNS : 1.59ns	WNS : -0.445ns	WNS : -0.09ns
				DRC Vio: 0	DRC Vio: 0	DRC Vio: 0	DRC Vio: 0	DRC Vio: 0
				Density: 0.782	Density: 0.789	Density: 0.789	Density: 0.85	Density: 0.85
				WNS : 0.703ns	WNS : 0.702ns	WNS : 0.703ns	WNS : 0.622ns	WNS : 0.622ns
				DRC Vio: 2	DRC Vio: 2	DRC Vio: 0	DRC Vio: 0	DRC Vio: 0
				Density: 0.664	Density: 0.665	Density: 0.667	Density: 0.667	Density: 0.667
				WNS : 5.62ns	WNS : 5.475ns	WNS : 5.470ns	WNS : 5.53ns	WNS : 5.53ns
				DRC Vio: 25	DRC Vio: 23	DRC Vio: 0	DRC Vio: 69	DRC Vio: 0
				Density: 0.62	Density: 0.624	Density: 0.628	Density: 0.624	Density: 0.625

power, at the cost of larger area. On VeriGPU, where OpenROAD is the only baseline, it reaches a comparable point with better TNS. Overall, an 8B vision-language backbone driven by our flow can match or surpass specialized macro placers on timing closure.

A.6 PDAgent Design Details

A.6.1 Agent Design

Each agent inherits from a common `BaseAgent` class that provides message handling, LLM access, tool query primitives, and a shared `DesignContext`. The `DesignContext` is a cached representation of the active design, populated by querying the EDA tool after initialization, including macro locations, die/core area, and clock topology. It is shared across all agents to maintain consistent design state throughout the optimization loop.

Worker Agent. The Worker Agent maintains a persistent EDA session and executes subtasks in three modes: (1) skill-guided generation, where the LLM synthesizes Tcl scripts from skill content and design context; (2) script execution, where reference Tcl scripts are sourced and run directly; and (3) optimization application, where targeted optimizations requested by the Optimizer Agent are applied and coordinated through the Planner Agent. Executed scripts, logs, and layout snapshots are organized by stage in a session history with manifest tracking, ensuring full traceability and enabling downstream agents to reference artifacts from any prior stage.

Analyzer Agent. The Analyzer Agent is selectively activated at analysis-critical stages (e.g., placement, CTS, and routing), where raw EDA tool logs are verbose and difficult to interpret directly. At each activated stage, it extracts a structured `StageMetrics` vector covering timing, utilization, routing, clock, and power metrics, and passes this condensed summary to the Debugger and Optimizer Agents for closed-loop refinement. Lightweight stages such as floorplanning and powerplanning bypass analysis, reducing runtime overhead without sacrificing result quality at critical design junctures.

Debugger Agent. After the Worker Agent completes a stage, the Debugger Agent checks for DRC violations and other errors by executing an iterative diagnosis loop. It loads the stage-specific debug skill, initializes a multi-turn LLM conversation with the skill content, EDA output, and convergence state as context, then iteratively performs check→fix→check cycles via direct tool queries. Each step is tracked in an `AgentStageContext` that maintains the full conversation history, violations found and fixed, and budget consumption. Skills provide the check criteria and corresponding fix strategies for each stage.

Table 10: Early-stage physical design results using a RAG-based method without skills on the PDAgent-Bench full flow suite (TSMC 28nm). Results cover the first three implementation stages: synthesis, floorplanning, and power planning. The RAG approach exhibits consistent failures across all stages, including setup errors, pin placement failures, and incomplete power networks, demonstrating its limited capability in automated physical design.

Design	init	floorplan	powerplan
TinyRISCV	5ns, WNS: 1.362ns Setup failed	Pin placement failed	Failed to use the correct layer
Open910	1.5ns, WNS: 0.465ns Setup failed	Pin placement failed	Failed to addRing
Systolic array	3.4ns, WNS: 1.11ns Setup successfully	Pin placement failed	DRC Vio: 346
16b pipelined mult.	2.6ns, WNS: 0.808ns Setup failed	Util: 0.65 DRC Vio: 0	Missing Bottom and Top Power Rings
AES-256	3.5ns, WNS: 1.003ns Setup failed	Pin placement failed	Stripes Not Connected to Power Ring
NVDLA-small	2ns, WNS: 0.65ns Setup failed	Pin placement failed	Missing Bottom and Top Power Rings
NVDLA-large	2.5ns, WNS: 0.69ns Setup failed	Pin placement failed	Missing Bottom and Top Power Rings
UART controller	3.4ns, WNS: 1.108ns Setup successfully	Util: 0.65 DRC Vio: 0	Stripes Not Connected to Power Ring
Elevator FSM	2.5ns, WNS: 0.708ns Setup successfully	Util: 0.7 DRC Vio: 0	Stripes Not Connected to Power Ring
Ethernet MAC	10ns, WNS: 5.621ns Setup failed	Pin placement failed	Missing Bottom and Top Power Rings

Table 11: Token usage of the full-flow implementation across all benchmark designs. We report input tokens, output tokens, the resulting total, and the number of LLM calls.

Design	Input	Output	Total	LLM calls
TinyRISCV	327,648	6,013	333,661	67
Open910	1,283,641	24,167	1,307,808	182
Systolic array	227,648	4,013	231,661	47
16b pipelined mult.	144,057	3,924	147,981	29
AES-256	259,783	3,948	263,731	57
NVDLA-small	1,259,783	21,948	1,281,731	137
NVDLA-large	1,595,980	25,943	1,621,923	192
UART controller	127,648	4,013	131,661	27
Elevator FSM	174,057	4,524	178,581	34
Ethernet MAC	444,057	8,924	452,981	79

To prevent unbounded iteration, a three-tier budget hierarchy is enforced: (1) micro-level, handling syntax errors with up to 2 retries; (2) meso-level, handling parameter adjustments with up to 4 retries; and (3) macro-level, handling cross-stage rollback with up to 2 retries. When the budget is exhausted, the Debugger Agent escalates to the Optimizer Agent for more aggressive intervention.

Optimizer Agent. The Optimizer Agent operates in two modes. In iterative mode, it conducts a multi-turn LLM-guided loop analogous to the Debugger Agent: checking current metrics, adjusting parameters (e.g., placement effort illustrated in Figure 2(c)), informed by stage-specific optimization skills and the full iteration trajectory stored in MetricsDB, then verifying improvements. In tree-of-thought mode, triggered on escalation or LLM-directed exploration, it performs branching design space exploration by generating multiple solution candidates and executing them either in parallel or sequentially, as described in §A.6.2.

Table 12: Macro placement results across all benchmark designs (our method as *Ours*).

Design	Method	Power ↓	WNS ↑	TNS ↑	Area ↓
ariane133	WireMask-EA	0.369	-0.417	-329.353	349228
	SA	0.365	-0.512	-650.399	347043
	MaskPlace	0.373	-0.349	-244.936	357322
	ChiPFormer	0.370	-0.553	-860.952	349005
	DREAMPlace	0.367	-0.540	-690.266	348180
	AutoDMP	0.350	-0.982	-1913.660	344091
	OpenROAD	0.359	-0.498	-596.718	352706
	Ours	0.257	-0.08	-4.01	357917
bp_be	WireMask-EA	0.467	-2.142	-4093.970	123966
	SA	0.459	-2.494	-5510.140	124667
	MaskPlace	0.456	-2.439	-4459.870	124938
	ChiPFormer	0.425	-2.169	-3541.820	110904
	DREAMPlace	0.458	-2.163	-3648.020	125221
	AutoDMP	0.453	-1.948	-3351.120	125471
	OpenROAD	0.409	-1.862	-2343.290	118786
	Ours	0.421	-0.63	-57.67	128865
bp_fe	WireMask-EA	0.309	-1.666	-777.420	77648
	SA	0.310	-1.179	-1236.880	81919
	MaskPlace	0.309	-1.318	-771.363	77881
	ChiPFormer	0.299	-1.197	-1000.190	72723
	DREAMPlace	0.294	-1.117	-473.261	77427
	AutoDMP	0.312	-1.253	-1198.430	81294
	OpenROAD	0.285	-1.092	-491.698	75884
	Ours	0.24	-0.43	-29.06	81323
swerv_wrapper	WireMask-EA	0.666	-1.027	-873.506	205627
	SA	0.648	-0.962	-854.737	200280
	MaskPlace	0.690	-1.251	-1306.460	206481
	ChiPFormer	0.674	-1.189	-1282.240	205775
	DREAMPlace	0.646	-1.061	-780.196	203896
	AutoDMP	0.648	-1.094	-773.339	200823
	OpenROAD	0.628	-1.127	-1163.820	202853
	Ours	0.626	-0.67	-750.37	204713
VeriGPU	OpenROAD	0.0779	-0.26	-16.84	271391
	Ours	0.0931	-0.28	-14.14	273414

A.6.2 Closed-Loop PPA Optimization Mechanism

The PPA Optimization Mechanism in PDAGent operates at two levels: a **stage-level** that progresses through the physical design flow step by step, and a **parameter-level** that explores the design space within each stage.

Stage-Level Execution. The Planning Agent maintains a plan $\mathcal{P} = \{s_1, \dots, s_n\}$ of subtasks ordered by dependency. For each subtask s_i , execution follows one of two paths based on the tool output:

(1) **Success path:** Tool $\xrightarrow{\text{result}}$ Planner $\xrightarrow{\text{if worthy}}$ Analyzer $\xrightarrow{\text{metrics}}$ Optimizer $\xrightarrow{\text{decision}}$ Planner $\rightarrow$ advance or re-run;

(2) **Failure path:** Tool $\xrightarrow{\text{error}}$ Planner $\rightarrow$ Debugger $\xrightarrow{\text{action}}$ Planner $\rightarrow$ {retry, fix, rollback, skip, escalate}. The convergence state, classified as improving, stalled, or diverging based on the metric trajectory across iterations, informs action selection at each decision point.

Parameter-Level Exploration. When the Optimizer Agent determines that local parameter adjustments are insufficient, for example upon escalation from the Debugger Agent or detection of a stalled convergence state, it activates a tree-of-thought search over the parameter space. The search proceeds in batches: at each batch $b \in \{1, \dots, B\}$, the LLM generates K candidate parameter configurations $\{c_1^{(b)}, \dots, c_K^{(b)}\}$, such as redoing macro placement or revising the power plan to reduce routing layer pressure, each accompanied by the corresponding EDA commands and evaluation queries. Each

You are a senior VLSI physical design engineer. Answer the user’s question with technically accurate, well-structured explanations. Cover key concepts, trade-offs, and any relevant tool/flow details. Be concise, compact but complete.

Figure 8: System prompt used for Physical Design Understanding tasks.

candidate is evaluated by saving a design checkpoint, applying the parameter modifications, executing the evaluation commands, and extracting the resulting metrics. The design is then restored to the checkpoint before the next candidate is evaluated.

The fitness of each candidate is computed as a stage-aware weighted score:

$$\text{score}(c) = \sum_{m \in \mathcal{M}_s} w_m^{(s)} \cdot \hat{v}_m(c), \quad (1)$$

where $\mathcal{M}_s$ denotes the metric set for stage s , $w_m^{(s)}$ the stage-specific weight, and $\hat{v}_m(c) \in [0, 1]$ the normalized metric value. The best candidate $c^* = \arg \max_c \text{score}(c)$ becomes the starting point for batch $b+1$. The search terminates when stage requirements are satisfied (e.g., DRC count = 0 for routing), the score improvement falls below threshold δ , or the batch budget B is exhausted.

Iteration Tracking. A persistent `MetricsDB` records the full parameter to metric mapping across all iterations, including parameter provenance, capturing which agent modified which parameter and the rationale behind each change. This trajectory data serves two purposes: it provides the Optimizer Agent with historical context for informed tuning decisions, and it enables post-hoc analysis of the DSO process for reproducibility and insight.

A.7 System Prompts

We list the system and judge prompts used throughout our evaluation in Figures 8–10. Figure 8 shows the system prompt provided to all evaluated models on the Physical Design Understanding tasks (Basic, RootOpt, Report, STA). Figure 9 shows the system prompt for the script-generation tasks (Innovus, ICC2, ECO, FM). Figure 10 shows the prompt given to our LLM-Judge (Claude Sonnet 4.6) for automatic scoring of free-response answers.

You are a senior VLSI physical design engineer. Answer each question with ONLY the exact Tcl/shell command(s) required. Do not add explanations, comments, markdown fences, or surrounding prose.

Figure 9: System prompt used at inference time for the script Generation tasks (Innovus, ICC2, ECO, FM).

Please act as an impartial physical-design engineer and judge the quality of the response provided by an AI assistant to the user question shown below. Evaluate the response objectively against the rubric supplied with each data item.
Begin your evaluation with a brief explanation justifying your assessment. Be as fair and objective as possible.
After the explanation, conclude with a numerical score from 0 to 10 in the following strict format:
score: X
For example: **score: 7**

Figure 10: LLM-Judge prompt used to score model responses on the Physical Design Understanding tasks.

A.8 Examples from PDAgent-Bench

We illustrate the structure and content of PDAgent-Bench through representative examples drawn from each task category. Figures 11–12 show a sample question, reference answer, and scoring rubric from the Basic benchmark. The complete benchmark, along with all model outputs and scoring results, is included in the supplementary material.

Question. In VLSI backend physical design flow, describe the essential input files required by a physical design tool and the key output files it generates. For each file, briefly explain its purpose and common format extensions.

Reference Answer.

Input data required for physical design:

1. **Technology file** (.tf in Synopsys, .techlef in Cadence): describes units, drawing patterns, layers, design rules, vias, and parasitic R/C of the manufacturing process.
2. **Physical libraries** (.lef, .gds; or Synopsys .CEL, .FRAM): layout information and abstract models for placement and routing (pin accessibility, blockages, etc.).
3. **Timing, logical, and power libraries** (.lib, or LM view .db): timing and power information for all design elements.
4. **TDF file** (.tdf / .io): pad/pin arrangement (order and location); for full-chip flows also captures VDD/VSS pads and power-cut diodes not present in the Verilog netlist.
5. **Constraints** (.sdc): area, power, and timing constraints.
6. **PDEF** (optional): row and cell placement locations.
7. **DEF** (optional): row, cell, and pre-existing placement information.

Output data from a physical design tool:

1. **Standard Delay Format** (.sdf): post-layout timing (excluding load information).
2. **Parasitics** (.spef, .dspf): extracted R/C of cells and nets.
3. **Post-routed netlist** (.v; flat or hierarchical): connectivity of all cells.
4. **Physical layout** (.gds): final layout for tape-out.
5. **Design Exchange Format** (.def): row, cell, and net placement locations.

Figure 11: Example question and reference answer from the Basic benchmark (Physical Design Understanding).

Evaluation Rubric (total: 10 points)

1. **Input Files — Technology & Libraries (3 pts).** 1 pt each for: (a) technology file (.tf / .techlef) and its role in process rules and parasitics; (b) physical libraries (.lef / .gds / .cel / .fram) and their role in layout abstraction; (c) timing/power libraries (.lib / .db) and their role in timing/power characterization.
2. **Input Files — Constraints & Optional Files (2 pts).** 1 pt each for: (a) SDC constraints file (timing, area, power); (b) IO/TDF or DEF/PDEF for pad placement or pre-existing placement data.
3. **Output Files — Timing & Parasitics (2 pts).** 1 pt each for: (a) SDF file for post-layout timing delays; (b) SPEF or DSPF for extracted RC parasitics.
4. **Output Files — Netlist, Layout & DEF (2 pts).** 1 pt for the post-routed Verilog netlist (.v, flat or hierarchical); 0.5 pt for GDS (physical layout); 0.5 pt for DEF (final placement and routing data).
5. **Clarity, Completeness & Technical Accuracy (1 pt).** Answer is well-organized (inputs vs. outputs clearly separated), file extensions correctly associated with descriptions, and no significant factual errors or omissions of major file types.

Figure 12: Scoring rubric corresponding to Figure 11, used by the LLM-Judge to grade model responses.

A.9 Privacy-Preserving Deployment with Multiple Commercial Parties

The agentic PD design workflow involves collaboration among commercial EDA tools, PDKs, and LLMs/VLMs, all of which must remain proprietary. To date, there are still no standards or policies to guide such design workflows. This creates major deployment and operability barriers for the community, as highlighted at the recent first Cognitive Silicon Design Workshop (CSDW) [36]. To maximally preserve privacy between these parties, our framework uses a Python coordinator that runs on local hardware/environment and mediates all interactions between commercial EDA tools, PDKs, and foundation models. Rather than allowing an LLM/VLM to invoke tools directly, the coordinator executes each tool command and writes its output to predefined disk locations. A deterministic parser then extracts only task-relevant fields, such as sign-off metrics (WNS, DRC count, and utilization) and compact error signatures. Consequently, the model is never exposed to raw tool dumps, foundry libraries, technology LEF/LIB files, or other proprietary artifacts. It receives only a narrow, sanitizable slice of information explicitly selected by the coordinator. This design makes the privacy boundary an enforceable software interface under our control, rather than relying on the model to prevent the leakage or misuse of sensitive information.